

PROCESS CONCEPTS

1.

```
sai_pavani@Sai-Pavani: ~/OS_ x + v
GNU nano 8.7.1 processid.c
#include<stdio.h>
#include<unistd.h>
int main(){
printf("My process ID (PID) = %d\n", getpid());
return 0;
}
```

```
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ nano processid.
c
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ gcc processid.c
-o processid
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ ./processid
My process ID (PID) = 3291
```

```
sai_pavani@Sai-Pavani: ~/OS_ x + v
GNU nano 8.7.1 fork1.c
#include <stdio.h>
#include <unistd.h>

int main(){
    pid_t pid = fork();
    if (pid == 0){
        printf("Child Process\n");
        printf("PID = %d\n", getpid());
    }
    else{
        printf("Parent Process\n");
        printf("PID = %d\n", getpid());
    }

    return 0;
}
```

```
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ nano fork1.c
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ gcc fork1.c -o
fork1
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ ./fork1
Parent Process
PID = 3503
Child Process
PID = 3504
```

```
sai_pavani@Sai-Pavani: ~/OS_ × + v
GNU nano 8.7.1 parentwaitsforchild.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(){

    pid_t pid = fork();

    if(pid==0){
        printf("Child Process Executing...\n");
        sleep(3);
        printf("Child Finished\n");
    }
    else {
        wait(NULL);
        printf("Parent process continues\n");
    }

    return 0;
}
```

```
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ nano parentwait
sforchild.c
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ gcc parentwaits
forchild.c -o parentwaitsforchild
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ ./parentwaitsfo
rchild
Child Process Executing...
Child Finished
Parent process continues
```

```
sai_pavani@Sai-Pavani: ~/OS_ × + v
GNU nano 8.7.1 multiplechildprocess.c *
#include <stdio.h>
#include <unistd.h>

int main(){

    for(int i=1;i<=3;i++){

        if(fork()==0){
            printf("Child %d PID = %d\n",i,getpid());
            return 0;
        }

    }

    return 0;
}
```

```
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ nano multiplech
ildprocess.c
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ gcc multiplechi
ldprocess.c -o multiplechildprocess
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ ./multiplechild
process
Child 1 PID = 3615
Child 2 PID = 3616
Child 3 PID = 3617
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ |
```

```
sai_pavani@Sai-Pavani: ~/OS_ × + ▾
GNU nano 8.7.1 using_exclp.c *
#include <stdio.h>
#include <unistd.h>

int main(){

    printf("Before exec()\n");
    execlp("ls","ls","-l",NULL);
    printf("This will not execute. \n");
    return 0;
}
```

```
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ nano using_exclp.c
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ gcc using_exclp.c -o using_exclp
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ ./using_exclp
Before exec()
total 120
-rwxr-xr-x 1 sai_pavani sai_pavani 16080 Sep  9 06:38 fork1
-rw-r--r-- 1 sai_pavani sai_pavani  285 Sep  9 06:37 fork1.c
-rwxr-xr-x 1 sai_pavani sai_pavani 16056 Sep  9 06:48 multiplechildprocess
-rw-r--r-- 1 sai_pavani sai_pavani  258 Sep  9 06:47 multiplechildprocess.c
-rwxr-xr-x 1 sai_pavani sai_pavani 16088 Sep  9 06:42 parentwaitsforchild
-rw-r--r-- 1 sai_pavani sai_pavani  381 Sep  9 06:42 parentwaitsforchild.c
-rwxr-xr-x 1 sai_pavani sai_pavani 16000 Sep  9 06:06 processid
-rw-r--r-- 1 sai_pavani sai_pavani  110 Sep  9 06:07 processid.c
-rwxr-xr-x 1 sai_pavani sai_pavani 16000 Sep  9 06:52 using_exclp
-rw-r--r-- 1 sai_pavani sai_pavani  190 Sep  9 06:51 using_exclp.c
-rwxr-xr-x 1 sai_pavani sai_pavani 16040 Aug 22 05:05 zombie
-rw-r--r-- 1 sai_pavani sai_pavani  183 Aug 22 05:07 zombie.c
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ |
```

```
sai_pavani@Sai-Pavani: ~/OS_ × + ▾
par_and_child_executing_concurrently.c *
#include <stdio.h>
#include <unistd.h>

int main(){
    fork();
    for(int i=1;i<=5;i++){
        printf("PID : %d Count: %d\n", getpid(),i);
        sleep(1);
    }

    return 0;
}
```

```

sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ nano par_and_chi
ld_executing_concurrently.c
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ gcc par_and_chi
ld_executing_concurrently.c -o par_and_child_executing_concurren
tly
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ ./par_and_child
_executing_concurrently
PID : 3812 Count: 1
PID : 3813 Count: 1
PID : 3813 Count: 2
PID : 3812 Count: 2
PID : 3812 Count: 3
PID : 3813 Count: 3
PID : 3812 Count: 4
PID : 3813 Count: 4
PID : 3812 Count: 5
PID : 3813 Count: 5
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$

```

zombie

```

GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
int main(){

    pid_t pid=fork();
    if(pid==0){
        printf("Child Exiting\n");
        return 0;
    }
    else{
        sleep(15);
        printf("Parent finished\n");
    }
    return 0;
}

```

```

sai_pavani@Sai-Pavani:~/OS_LAB$ cd process_concepts
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ nano zombie.c
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ gcc zombie.c -o zombie
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ ./zombie
Child Exiting
Parent finished

```

```
sai_pavani@Sai-Pavani: ~/OS_ × + ▾
GNU nano 8.7.1 orphan_process.c *
#include <stdio.h>
#include <unistd.h>

int main(){

    pid_t pid = fork();

    if(pid > 0){

        printf("Parent Exiting\n");
        return 0;

    }
    else{

        sleep(5);
        printf("Child PID : %d\n",getpid());
        printf("New Parent PID : %d\n",getppid());

    }

    return 0;

}
```

```
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ nano orphan_pro
cess.c
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ gcc orphan_proc
ess.c -o orphan_process
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ ./orphan_proces
s
Parent Exiting
sai_pavani@Sai-Pavani:~/OS_LAB/process_concepts$ Child PID : 386
2
New Parent PID : 318
```